Bibliophile Analytics Service

Technical Report

Author:

Basmala Saeed

March 2026

1. Project Overview

The Bibliophile Analytics Service is a standalone Python microservice that provides deep analytics, interactive dashboards, PDF reports, and an AI-powered recommendation engine for the Bibliophile API platform.

It runs independently on port 8000 alongside the main Node.js/TypeScript backend on port 5000, communicating with it via internal HTTP calls. All analytics operations are read-only — the service never writes to the main application database except for the recommendation history collection.

Key Capabilities

Capability	Description
Books Analytics	Category, author, language, decade, and rating distributions across 9,003 books
Subscription Analytics	Revenue, retention, plan distribution, gender/role breakdowns for 1,312 subscriptions
Interactive Dashboards	Plotly-powered HTML dashboards with 4 theme options and real-time filtering
PDF Reports	ReportLab-generated downloadable reports for all entities
Recommendation Engine	Content-based filtering, collaborative filtering, and Gemini AI recommendations
Recommendation History	MongoDB-persisted history with aggregated analytics
In-Memory Caching	TTL-based cache to reduce redundant API calls to Node backend
Pagination	Book search supports page/limit parameters for efficient data loading

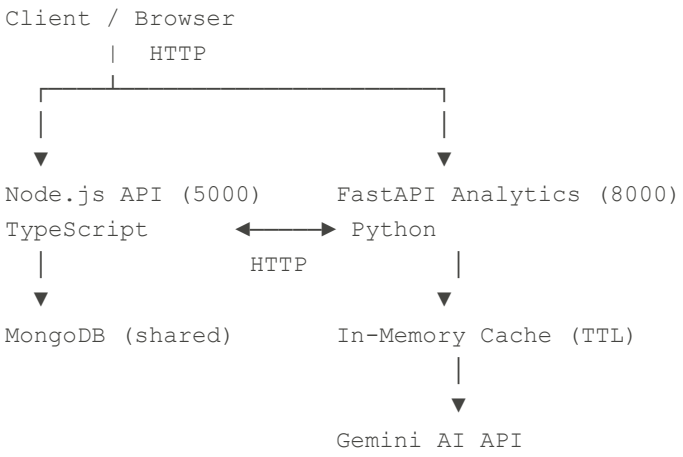
---

2. Architecture

2.1 System Architecture

The analytics service operates as a sidecar microservice to the main Bibliophile API. It fetches data from the Node.js backend via HTTP, processes it in Python, and serves HTML dashboards, JSON endpoints, and PDF downloads.

Architecture Overview



2.2 Directory Structure

File / Directory	Purpose
app/core/config.py	Environment variable management via Pydantic Settings
app/core/cache.py	In-memory TTL cache singleton
app/db/database.py	MongoDB Motor async connection
routers/dashboard_router.py	Main dashboard + cache management endpoints
routers/books_router.py	Books, search, analytics endpoints
routers/subscriptions_router.py	Subscriptions, explorer, report endpoints
routers/plans_router.py	Plans analytics endpoints
routers/recommendations_router.py	Full recommendation engine endpoints
services/book_service.py	Books fetching, search, pagination + cache integration
services/analytics_service.py	Core analytics calculations for all entities
services/chart_service.py	Matplotlib static chart generation for PDFs
services/plotly_service.py	Main Plotly interactive dashboard
services/search_plotly_service.py	Entity dashboards + subscription explorer
services/pdf_service.py	ReportLab PDF generation for all report types
services/search_analytics_service.py	Per-entity analytics (category, author, plan)
services/user_book_service.py	User-book data fetching and enrichment
services/recommendation_service.py	TF-IDF content-based + collaborative filtering
services/openai_service.py	Gemini AI integration for recommendations

File / Directory	Purpose
services/recommendation_plotly_service.py	HTML dashboards for recommendations
services/recommendation_history_service.py	MongoDB CRUD for recommendation history

---

3. Tech Stack

Layer	Technology	Purpose
Framework	FastAPI	REST API framework with automatic OpenAPI docs
Server	Uvicorn	ASGI server with hot-reload for development
HTTP Client	HTTPX	Async HTTP calls to Node.js backend
Database	Motor (AsyncIOMotorClient)	Async MongoDB driver for Python
ML	scikit-learn	TF-IDF vectorization and cosine similarity
ML	NumPy	Matrix operations for collaborative filtering
Visualization	Plotly	Interactive HTML charts and dashboards
Visualization	Matplotlib	Static chart generation for PDF reports
PDF Generation	ReportLab	Professional PDF document creation
AI Integration	Google Generative AI	Gemini 2.0 Flash for AI recommendations
Config	Pydantic Settings	Type-safe environment variable management

3.1 Environment Variables

```
NODE_BACKEND_URL=http://localhost:5000
GEMINI_API_KEY=your_gemini_key_here
MONGO_URI=your_mongodb_uri_here
MONGO_DB=bibliophile
```

3.2 Dependencies (requirements.txt)

```
fastapi, uvicorn, httpx, pydantic-settings
motor, matplotlib, reportlab, Pillow
plotly, kaleido, numpy, scikit-learn
google-generativeai
```

---

4. Database Schema

4.1 recommendation_history Collection

This is the only collection owned by the analytics service. All other MongoDB collections are read-only via the Node.js API.

Field	Type	Description
user_id	string	MongoDB ObjectId of the user who requested recommendations
user_name	string	Full name of the user (firstName + lastName)
requested_at	ISODate	UTC timestamp of the recommendation request
books_read_count	integer	Number of books in the user's library at time of request
top_categories	string[]	Up to 5 of the user's most common book categories
methods_used	string[]	Which methods were used: content_based, collaborative, openai
total_recs	integer	Total number of recommendations generated across all methods
recommendations	object[]	Array of recommended book objects (see sub-fields below)
recommendations.method	string	Which method generated this recommendation
recommendations.book_id	string	MongoDB ObjectId of the recommended book
recommendations.title	string	Title of the recommended book
recommendations.categories	string[]	Categories of the recommended book
recommendations.score	float | null	Similarity score (null for Gemini AI recommendations)

4.2 Node.js Collections Used (Read-Only via HTTP)

Collection	Records	Used For	Node.js Endpoint
books	9,003	All book analytics, recommendations	GET /api/v1/books
subscriptions	1,312	Subscription analytics, explorer	GET /api/v1/subscriptions
plans	3	Plan analytics and comparison	GET /api/v1/plans
user-books	1,573	Recommendation engine input data	GET /api/v1/user-books

---

5. API Endpoints Reference

5.1 Dashboard Endpoints

Method	Endpoint	Description
GET	/dashboard/interactive	Main Plotly dashboard — books, plans, subscriptions
GET	/dashboard/report	Full PDF report download
GET	/dashboard/preview	JSON analytics summary
GET	/dashboard/cache/status	Show active cache keys and TTL
GET	/dashboard/cache/clear	Clear all cached data (forces fresh fetch)

5.2 Books Endpoints

Method	Endpoint	Key Parameters
GET	/books/	—
GET	/books/search	title, author, category, page, limit
GET	/books/analytics/category	name, theme, format (html|pdf)
GET	/books/analytics/author	name, theme, format (html|pdf)
GET	/books/analytics/book	id, theme, format (html|pdf)

Method	Endpoint	Key Parameters
GET	/books/analytics/unknown-authors	theme, format (html|pdf)

5.3 Subscriptions Endpoints

Method	Endpoint	Description
GET	/subscriptions/	Sanitized subscriptions list (no PII)
GET	/subscriptions/analytics	Full subscription analytics JSON
GET	/subscriptions/dashboard	Interactive filterable explorer with subscribers table
GET	/subscriptions/report	Subscription PDF report download
GET	/subscriptions/data	Lightweight data endpoint for frontend use

5.4 Plans Endpoints

Method	Endpoint	Key Parameters
GET	/plans/	—
GET	/plans/analytics	—
GET	/plans/analytics/plan	name, theme, format (html|pdf)

5.5 Recommendations Endpoints

Method	Endpoint	Key Parameters
GET	/recommendations/user/{user_id}	method, top_n, format (json|html), theme
GET	/recommendations/book/{book_id}	method, top_n, format (json|html), theme
GET	/recommendations/analytics	top_n (10-200)
GET	/recommendations/analytics/dashboard	theme, top_n

Method	Endpoint	Key Parameters
GET	/recommendations/history/user/{user_id}	limit
GET	/recommendations/history/analytics	—

method parameter options: content | collaborative | openai | all

---

6. ML Models & Algorithms

6.1 Content-Based Filtering

Goal: Recommend books similar to what the user has already read, based on content features extracted from each book.

Feature Engineering

Each book is represented as a text string combining categories (weighted 2x), authors, and title:

```
features[book_id] = f"{categories} {categories} {authors} {title}"
```

Categories are repeated twice to give them higher TF-IDF weight, as they are the most reliable signal for content similarity. Author names and titles provide additional specificity.

Algorithm Steps

- Build a feature string for all 9,003 books in the catalog
- Apply TfidfVectorizer with English stop-word removal
- For the target user, extract TF-IDF vectors for all books they have read
- Compute the mean vector across all read books — this represents the user's taste profile
- Compute cosine similarity between the user vector and all book vectors
- Filter out books already read by the user
- Return top-N books sorted by similarity score (descending)

Similarity Score: Ranges from 0.0 (no similarity) to 1.0 (identical content). In practice, scores between 0.15 and 0.50 are typical for relevant recommendations given the diversity of the book catalog.

6.2 Collaborative Filtering

Goal: Find users with similar reading patterns and recommend books they have read that the current user has not yet discovered.

Algorithm Steps

- Build a binary User-Item matrix: rows = users (1,573), columns = books (9,003), value = 1 if user has the book
- Compute cosine similarity between the target user's row vector and all other users
- Select the top-20 most similar users
- For each similar user, accumulate recommendation scores for books the current user has not read, weighted by user similarity
- Sort accumulated scores and return top-N book recommendations

Matrix Dimensions: 1,573 users x 9,003 books. The matrix is sparse (most users have 1-8 books), making cosine similarity between users an effective measure of taste alignment.

6.3 Gemini AI Recommendations

Goal: Generate contextual, personalized book recommendations using large language model reasoning beyond what structured similarity metrics can provide.

Property	Value
Model	gemini-2.0-flash
API Provider	Google Generative AI
Input	User name + up to 10 read books with categories + top interest categories
Output	JSON array: title, author, category, reason
Quota Handling	Returns empty list with clear message on 429 errors — never crashes
Fallback	Main request always succeeds even if Gemini fails

6.4 Data Preprocessing

All book data goes through a cleaning pipeline before being used in any ML operation. Books with missing or unknown author/category data are handled gracefully:

```
UNKNOWN_NAMES = {"unknown author", "unknown", ""}
```

The _extract_str_list() function filters out all unknown/empty values and handles both string and dict formats from the API response, ensuring clean feature vectors for all ML operations.

---

7. Dashboards & Visualizations

7.1 Theme System

All dashboards support 4 themes via the ?theme= query parameter:

Theme	Primary Color	Style
default	#534AB7 — Purple	Light background, purple accents
dark	#7F77DD — Light Purple	Dark background, soft purple accents
ocean	#185FA5 — Blue	Light background, ocean blue accents
sunset	#D85A30 — Orange	Light background, warm sunset accents

7.2 Main Dashboard (/dashboard/interactive)

Section	Charts Included
Books Overview KPIs	Total books, premium %, free books, avg rating, with-PDF count
Top Categories	Horizontal bar chart — top 15 categories by book count
Premium vs Free	Donut pie chart with percentage labels
Language Distribution	Donut pie chart — top 10 languages
Books by Decade	Bar chart — publication decade distribution
Rating Distribution	Bar chart — 5 rating buckets
Top Authors	Horizontal bar chart — top 15 authors by book count
Subscription KPIs	Total, active, retention rate, total revenue
Monthly Subscription Growth	Bar + cumulative line combo chart
Plan Distribution	Donut pie chart — subscribers per plan

7.3 Entity Dashboards

Dashboard	Endpoint	Contents
Category	/books/analytics/category	KPIs, top authors bar, decade bar, rating bar, premium pie, top rated books grid
Author	/books/analytics/author	KPIs, categories bar, decade bar, rating bar, premium pie, top rated books grid
Book Detail	/books/analytics/book	Cover image, full metadata, description, preview link
Unknown Authors	/books/analytics/unknown-authors	Books with missing author info — same layout as author dashboard
Plan	/plans/analytics/plan	KPIs, monthly growth combo chart, gender pie, role pie

7.4 Subscription Explorer (/subscriptions/dashboard)

An interactive filterable dashboard where all charts and KPIs update in real time as filters are applied.

Feature	Details
Filters	Plan, Gender, Role, Status (Active/Inactive), Date range (month)
Dynamic KPIs	Total, Active, Revenue, Retention %, Avg Revenue/Sub
Charts	Monthly Growth with cumulative line, Plan Distribution pie, Gender pie, Role Distribution bar
Subscribers Table	Top 100 of filtered results — Name, Plan, Gender, Role, Status, Paid At
Recommendations Link	Every subscriber row has a View Recs button linking to their recommendation dashboard

7.5 Recommendation Dashboards

Dashboard	Endpoint	Key Features
User Recommendations	/recommendations/user/{id}?format=html	KPIs, categories chart, similarity scores chart, books read grid, 3 tabbed recommendation methods
Book Similar	/recommendations/book/{id}?format=html	Source book hero, similarity scores chart, 2 tabbed recommendation methods

Dashboard	Endpoint	Key Features
Recommendation Analytics	/recommendations/analytics/dashboard	System-wide KPIs, score distribution, method breakdown pie, top categories bar, most recommended books table

All ML recommendation cards are clickable — clicking any card navigates to that book's detail dashboard.

7.6 PDF Reports

Report	Endpoint	Contents
Main Report	/dashboard/report	Books KPIs, all charts, plans comparison table, subscription breakdown
Category Report	/books/analytics/category?format=pdf	Category KPIs, top authors table, top rated books, 4 charts
Author Report	/books/analytics/author?format=pdf	Author KPIs, categories table, top rated books, 4 charts
Plan Report	/plans/analytics/plan?format=pdf	Plan details, subscriber KPIs, demographics table, 3 charts
Subscription Report	/subscriptions/report	Subscription KPIs, plan distribution table, demographics, 3 charts

---

8. Caching System

8.1 Design

A simple in-memory key-value store with TTL (Time To Live) is implemented as a singleton in app/core/cache.py. It exposes five methods: get, set, delete, clear, and keys.

8.2 Cache Configuration

Cache Key	TTL	Cached Data
all_books	600 seconds (10 min)	All 9,003 books from Node.js API

8.3 Performance Impact

Without caching, every recommendation request for a single user fetches all 9,003 books from the Node.js API. The table below shows the difference:

Scenario	Response Time	Node API Calls
First request (cache miss)	~2,000ms	1 call — fetches and caches
Subsequent requests within TTL	~50ms	0 calls — served from cache
After TTL expires	~2,000ms	1 call — refreshes cache

8.4 Cache Management Endpoints

- GET /dashboard/cache/status — returns list of currently cached keys
- GET /dashboard/cache/clear — clears all cached data and forces fresh fetch on next request

---

9. Recommendation History

9.1 Purpose

Every time a user requests recommendations, the full request context and results are persisted to MongoDB. This enables historical review, usage analytics, and system improvement insights.

9.2 Storage Strategy

The save operation is fire-and-forget — it runs asynchronously after the main response is already prepared and sent to the user. This ensures:

- The user always receives their recommendations immediately
- A failed database write never impacts the recommendation response
- The history is eventually consistent without blocking the critical path

```
try:
    await save_recommendation(...)
except Exception:
    pass  # Never blocks the main response
```

9.3 History Analytics

The /recommendations/history/analytics endpoint returns aggregated MongoDB aggregation pipeline results:

Field	Description
total_requests	All-time count of recommendation requests
top_users	Top 10 users by recommendation request frequency
top_categories	Most common user interest categories across all requests
daily_requests	Request volume per day for the last 30 days

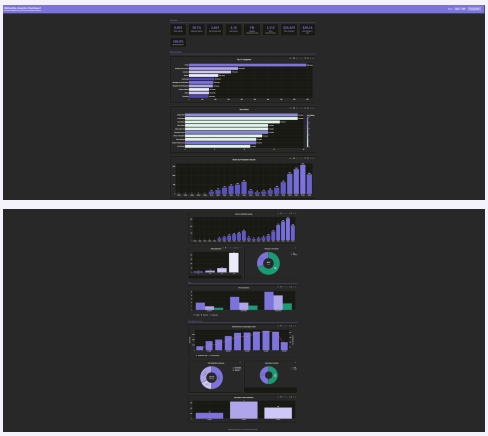
---

10. Screenshots

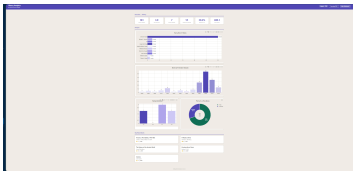
The following placeholders should be replaced with screenshots from the running analytics service.

10.1 Main Dashboard

Theme	URL	Screenshot
Default	/dashboard/interactive?theme=default	

Theme	URL	Screenshot
Dark	/dashboard/interactive?theme=dark	
Ocean	/dashboard/interactive?theme=ocean	

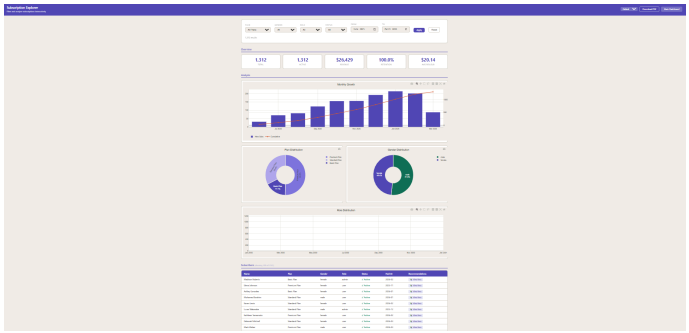
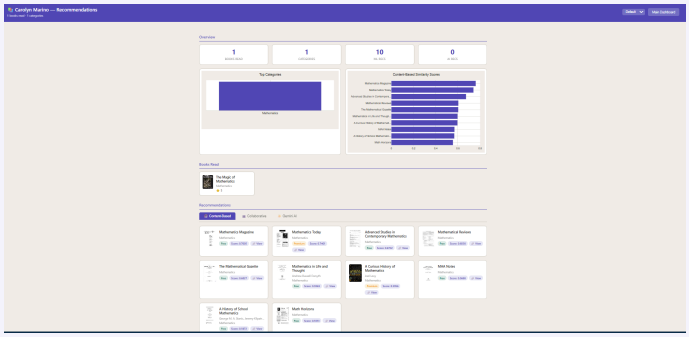
10.2 Books Analytics

Dashboard	URL	Screenshot
Category — Fiction	/books/analytics/category?name=Fiction	
Author Dashboard	/books/analytics/author?name={name}	

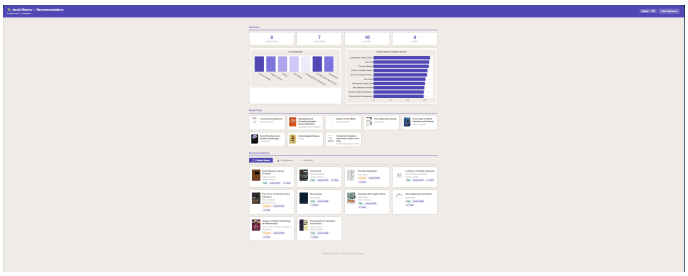
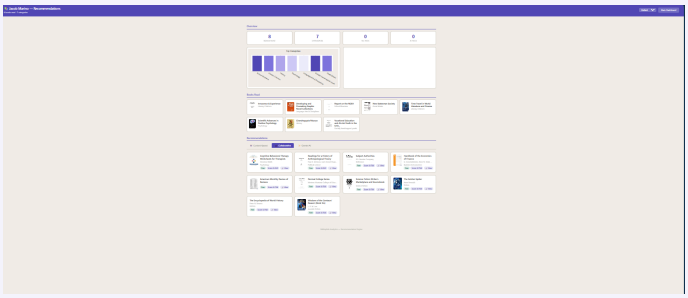
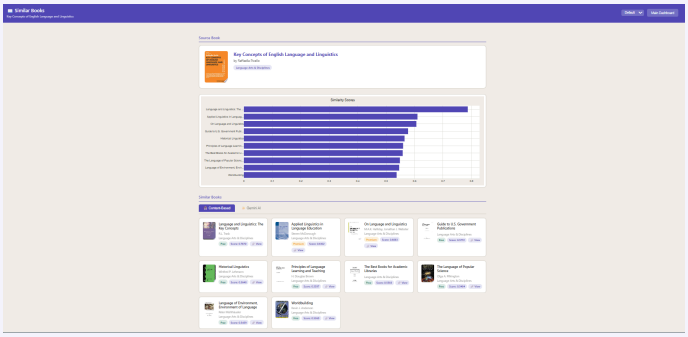
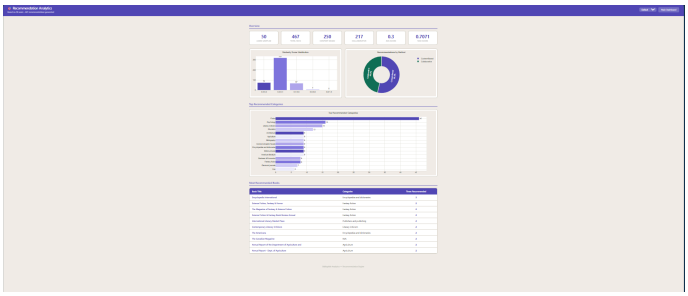
Dashboard	URL	Screenshot
Book Detail	/books/analytics/book?id={id}	

10.3 Subscription Explorer

URL: /subscriptions/dashboard?theme=default — show the subscribers table with View Recs buttons visible.

View	Screenshot
Subscription Explorer — Charts View	
Subscription Explorer — Subscribers Table	

10.4 Recommendation Dashboards

Dashboard	Screenshot
User Recommendations — Content-Based tab	
User Recommendations — Collaborative tab	
Book Similar Dashboard	
Recommendation Analytics Dashboard	

10.5 PDF Reports

Report	Screenshot

Main Dashboard PDF	

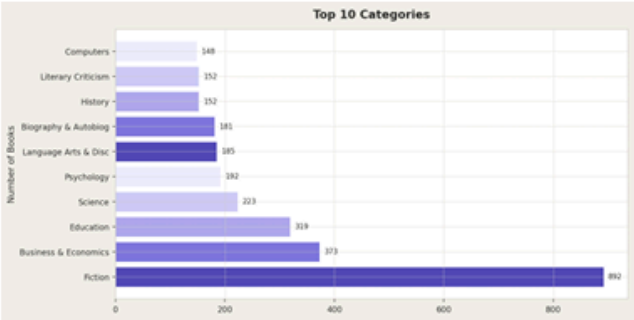
Bibliophile API

Analytics Report • March 21, 2026

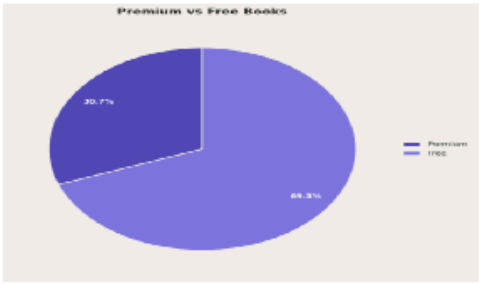
Books Overview

Total Books	Premium	Free	Avg Rating	With PDF
9003	2763 (30.7%)	6240	4.16	0 (0.0%)

Top Categories

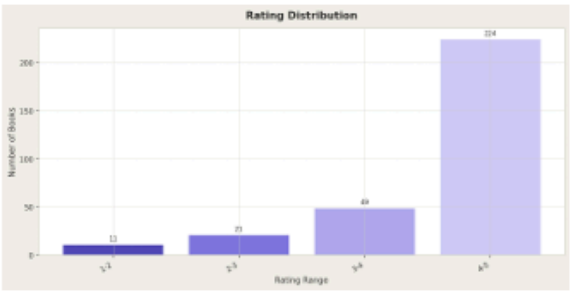


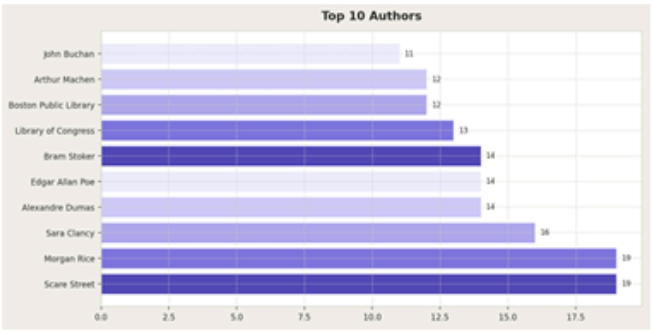
Premium vs Free



Books by Decade

Rating Distribution

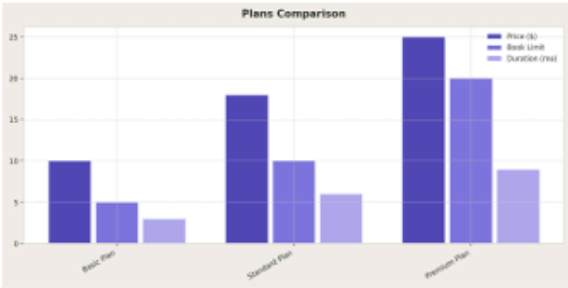




Subscription Plans

Plan	Price	Book Limit	Duration	$/Book	$/Month	Value
Basic Plan	$10.0	5	3 mo	$2.0	$3.33	1.5
Standard Plan	$18.0	10	6 mo	$1.8	$3.0	3.33
Premium Plan	$25.0	20	9 mo	$1.25	$2.78	7.2

Best value plan: **Premium Plan**



Subscriptions

Total	Active	Retention	Total Revenue	Avg Rev/Sub	Avg Duration
1312	1312	100.0%	$26,429	$20.14	211.5 days

Plan Distribution

Plan	Subscribers	Revenue	Percentage
Premium Plan	659	$16,475	50.2%
Standard Plan	428	$7,704	32.6%
Basic Plan	225	$2,250	17.1%

Demographic Breakdown

Gender	Count
female	635
male	677

Role	Count
admin	144
user	1168

Monthly Subscription Growth

Bibliophile Analytics Service • 2026-03-21 18:01